

SynBioHub: A Standards-Enabled Design Repository for Synthetic Biology

James Alastair McLaughlin,[†] Chris J. Myers,[‡] Zach Zundel,[‡] Göksel Mısırlı,[§] Michael Zhang,[‡] Irina Dana Ofiteru,^{||} Angel Goñi-Moreno,[†] and Anil Wipat^{*,†}

[†]School of Computing, Newcastle University, Newcastle upon Tyne, NE1 7RU, U.K.

[‡]Department of Electrical and Computer Engineering, University of Utah, Salt Lake City, Utah 84112, United States

[§]School of Computing and Mathematics, Keele University, Newcastle, ST5 5BG, U.K.

^{||}School of Engineering, Newcastle University, Newcastle upon Tyne, NE1 7RU, U.K.

ABSTRACT: The SynBioHub repository (<https://synbiohub.org>) is an open-source software project that facilitates the sharing of information about engineered biological systems. SynBioHub provides computational access for software and data integration, and a graphical user interface that enables users to search for and share designs in a Web browser. By connecting to relevant repositories (e.g., the iGEM repository, JBEI ICE, and other instances of SynBioHub), the software allows users to browse, upload, and download data in various standard formats, regardless of their location or representation. SynBioHub also provides a central reference point for other resources to link to, delivering design information in a standardized format using the Synthetic Biology Open Language (SBOL). The adoption and use of SynBioHub, a community-driven effort, has the potential to overcome the reproducibility challenge across laboratories by helping to address the current lack of information about published designs.

KEYWORDS: Synthetic Biology Open Language, SBOL, repository, web of registries, data standards, database, sharing



A fundamental goal of synthetic biology is to make biological systems easier to engineer.¹ As part of this endeavor, significant attention is being paid to the development of workflows^{2–6} that will assist researchers through the synthetic biology lifecycle. By drawing on concepts from engineering,⁷ an emphasis is placed on the design process through the use of basics such as standardization, reusability, abstraction, modularity, and predictability. Numerous so-called genetic design automation (GDA) tools and techniques are now emerging to aid in the design of engineered biological systems,^{8–10} e.g., TinkerCell,¹¹ SBOLDesigner,¹² iBioSim,¹³ SBROME,¹⁴ CELLO,¹⁵ Pigeon,¹⁶ DNAplotlib,¹⁷ and VisBOL.¹⁸ Nevertheless, designing new engineered organisms is still difficult due to the complexity of these systems and gaps in our knowledge of the biology involved.

One of the biggest challenges, and a barrier to the reuse of successful designs, is that biological data relevant to the design of novel systems are often not exchanged. Design efforts are usually carried out in individual laboratories, by teams in different geographic locations. The lack of data exchange not only concerns the genetic details and sequence of the designed system, but also experimental data about the characterization and behavior of the resulting system and mathematical models. As a result, it is very difficult, if not impossible, to repeat or reuse the work presented in many synthetic biology publications.¹⁹

To address this problem, several systems for storing and sharing data about engineered biological designs have been developed. Most notable are JBEI-ICE,²⁰ the iGEM Registry of Standard Biological Parts (<http://parts.igem.org>), and the

SBOL Stack.²¹ Many commercial and noncommercial software tools also exist that offer the ability to store genetic designs, such as VectorNTI,²² Benchling, and Addgene.²³ JBEI-ICE is one such registry that allows information about biological parts to be managed and shared. The repository is accessible via both a Web browser and through a Web application programming interface (API). ICE also introduced the concept of a Web of Registries, providing support for ICE repositories to be connected to allow distributed and interconnected use. The ICE repository supports the import and export of sequence data in various formats, but it is limited in its support of other design information, such as non-DNA components and their interactions. Another example repository is the SBOL Stack, which provides a database for programatically storing design information, but is not a “user-friendly” solution suitable for users outside of computer science.

Despite these developments in the storage of DNA designs, there is an increasing need to capture information about synthetic systems at a more abstract level, including information about proteins, interactions, metabolites, and the organism for which a design is intended to operate in once implemented. It is also very important to capture the history of design, so that the lineage of designs can be recorded. The inclusion of this type of information requires richer data standards beyond the simple GenBank and EMBL formats that are still used routinely in the design of engineered biological

Received: November 11, 2017

Published: January 9, 2018



Figure 1. Front page of SynBioHub. Users can choose to search for, upload, or share designs for publication or collaboration.

systems. The Synthetic Biology Open Language (SBOL) was developed to meet this need.^{24,25}

The initial version of SBOL focused upon exchanging genetic descriptions of parts at the DNA level, while SBOL version 2^{25,26} supports other types of genetic circuit components, including proteins, RNAs, and small molecules. The standard also supports provenance information²⁷ concerning lineage and history of a design, together with mechanisms that allow integration with other data standards that allow laboratory experimental data to be included with the design for a system. In short, SBOL now allows the design for a system to be shared, together with information on the purpose and history of that design, mathematical models of the design, visual representations, and experimental characterization data (captured in other complementary data standards) that describe the behavior of the system once built and tested. Finally, SBOL also includes a visualization standard, SBOL Visual, for representing genetic circuit designs in diagrams composed of a standard set of glyphs representing genetic components.²⁸

The continued development of SBOL provides the opportunity for a design repository specifically for synthetic biology, which can capture data not just about sequences on a DNA level, but information about the *who*, *where*, *why*, and *how* of the design process. However, so far there exists no central “hub” to deposit SBOL designs while preserving the capability to search for, retrieve, and visualize the complex information captured in the design process. Although the JBEI-ICE repository can import and export SBOL data files, its internal data model is still organized around a sequence-centric representation. In other words, JBEI-ICE stores SBOL as an

opaque file and the rich information available in SBOL is not exposed to the user for querying or in a form that can be integrated with other data, such as experimental data and mathematical models. Therefore, a need exists for a data repository that can capture this genetic design information encoded in SBOL 2 in a more computationally tractable form.

To address this need, we have developed SynBioHub, a repository for storing information about engineered biological systems, built on the SBOL Stack database and using SBOL 2 as the core storage format. SynBioHub functions both as a place to store designs, and as an aggregation facility to bring together information from many different repositories, including JBEI-ICE, Benchling, and the iGEM Registry of Standard Biological Parts. The syntactic and semantic disparity between these repositories is automatically harmonized in SynBioHub through conversion to and from SBOL, which makes it easier to search and retrieve design components with a canonicalized syntax. Since designs are exchanged and stored in the form of SBOL, the repository is also capable of storing information about the provenance of designs; designs for which the sequence is not yet available; and designs that include information about RNA, proteins, interactions, metabolites, and mathematical models.

SynBioHub provides a Web-based user interface allowing users to browse, upload, and share designs without the requirement of any locally installed software. Unlike the original SBOL Stack, SynBioHub completely abstracts away the details of the underlying data representation, enabling nontechnical users to interact with the front-end while

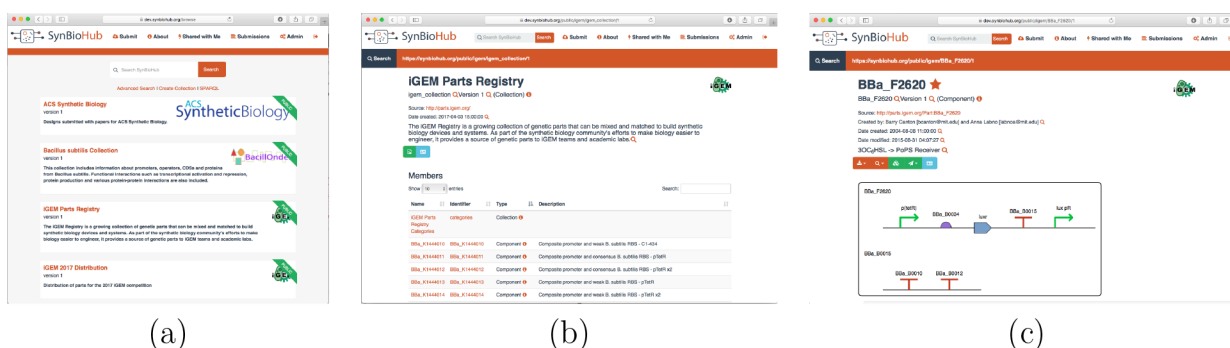


Figure 2. Search and data visualization in SynBioHub. (a) The browse design collections page shows all of the “root” collections in the repository. These are collections that are not contained by any other collection. The reference instance includes, among others, collections for designs published in ACS Synthetic Biology, a collection of *Bacillus subtilis* parts, and the iGEM registry parts. (b) The iGEM Parts Registry collection on the reference instance of SynBioHub. This collection view shows all of the parts available in the selected collection. The parts can be sorted by name, identifier, type, or description, and can be filtered using a free-text search. (c) The SynBioHub part view showing a part from the iGEM Registry. The SBOL Visual depiction is generated using VisBOL by mapping Sequence Ontology terms to glyphs. The part view can be used to send parts to the Benchling sequence design tool or to an ICE repository.

generating semantically rich SBOL data to facilitate data integration and analysis.

SynBioHub also supports the Web of Registries concept: the idea of multiple separate repositories linked together by shared common semantics. In the case of SynBioHub, SBOL is used to support the common exchange of data, thus positioning SynBioHub to support the development of synthetic biology workflows by acting as a source and a storage facility for designs. SynBioHub demonstrates the application of this harmonized data exchange through integration with existing repositories. We envisage a wider, integrated ecosystem of biological part information shared across a plethora of different repositories with different capabilities.

RESULTS AND DISCUSSION

The architecture of SynBioHub is designed to accommodate the concept of FAIRdom previously applied to systems biology:²⁹ that data should be findable, accessible, interoperable, and reusable. For SynBioHub, this means that powerful querying facilities should be available for findability; both a user interface and computational API should be available for accessibility; design data should be represented using an open, standardized data model for interoperability; and all information possible should be preserved to facilitate reuse. To accomplish these goals, the front page of any SynBioHub installation provides three options: to search for designs, upload designs to the repository, and share existing uploaded designs for publication or collaboration (Figure 1). Each of these three major functionalities is described in more detail below.

Search and Data Visualization. To facilitate efficient search capabilities, SynBioHub provides a flexible data storage mechanism for storing SBOL data. These data are graph based and include information about biological design components, their sequences, and interactions between these components. Designs are often hierarchical; a parent component can be formed of several child components which in turn may include complex designs. Behavioral representations of biological systems are modularized through parts that are defined with inputs and outputs to scale up to larger designs. These complex relationships are best represented as graphs. As SBOL is based on RDF/XML, it already takes advantage of graph representations. Although different options can be considered when storing data in a database, we chose a graph-based

database approach to take advantage of the storage and integration of SBOL documents, and also the efficient querying of data and underlying complex relationships.

To achieve this goal, the central component of SynBioHub is the SBOL Stack²¹ architecture. The SBOL Stack pioneered the concept of using an RDF triplestore to store SBOL data as triples, and retrieving designs in whole or part using SPARQL queries.³⁰ SynBioHub takes this further by providing a complete user-facing Web interface. Behind the scenes, queries to the Web interface are translated into SPARQL queries using the SBOL data model, and data are returned from the triplestore as RDF triples. These triples are then used to construct a partial graph, which is queried to produce information listings and visual depictions using VisBOL.¹⁸ The database is effectively schemaless. Therefore, SBOL documents that include custom and complex user annotations can also be stored efficiently, and queries can be extended to retrieve custom information. As data relevant to the design of biological systems grow, this approach makes the integration and retrieval of data a streamlined process in synthetic biology.

While querying the RDF triplestore using SPARQL queries provides a powerful means for search, constructing such queries for most target users would prove quite challenging. Therefore, SynBioHub provides several simplified means of search. For example, a user can perform simple text based searches that identify designs that include the provided terms within their identifier, name, or description. SynBioHub also includes a more advanced search capability that allows designs to be filtered by the date they were created, the name of their creator, their type and functional role, *etc.* Finally, a user can simply browse the designs organized into public collections (Figure 2a), and upon selecting a collection, the user is presented with a collection view that allows the search to be further refined (Figure 2b).

Once a design is selected, either from browsing collections or from search results, the user is presented with the Design View (Figure 2c). This view renders the design information data captured by SBOL in a human readable form. At the top of the page, the title, version, type, modification dates, short description, and provenance information (*i.e.*, where the part was retrieved from) are displayed. Next, there is an SBOL Visual representation of the design rendered using VisBOL.¹⁸ If the features shown in this visual depiction point to another

design in the Web of Registries, they can be selected to navigate to the view for that design. Below the visual depiction, more detailed information is provided, such as Sequence Ontology³¹ roles, custom user-defined annotations, data files, and other attachments.

Submission and Design Documentation. Registered users of SynBioHub can submit new designs *via* the submission interface. New designs are submitted into collections that are in the private repository space for the logged in user. Therefore, these submitted designs are initially only visible to the users who submit them. Methods for the sharing of this design information with other users and the public are described below. Each new design document can either be uploaded into a new private collection or merged with an existing private collection. SynBioHub further simplifies the uploading of design data by supporting a variety of data formats, including older SBOL versions, GenBank, and FASTA. SynBioHub utilizes libSBOLj,³² a Java library developed to read and write compliant SBOL documents, when converting different formats to the most recent SBOL version.

Once uploaded, additional information can be added to further document a design. In particular, the design view also incorporates “wiki” functionality, by which a much longer description can be edited by multiple users, subject to the permission model described below. References to publications can also be added by entering PubMed Ids that are then used to construct full references. Finally, SynBioHub allows attachments to be added to any design. These attachments can be of any file format, such as an SBML model, a graph image, or a spreadsheet. Attachments are saved in a compressed file store and can be downloaded on demand by any user with access to the design. Image attachments can be displayed within the “wiki” description.

Sharing and Security. As mentioned above, a newly submitted design is initially placed in the private repository for the submitting user. This private repository is a distinct, private graph in Virtuoso, the RDF triplestore database. To ensure the data in this graph remains private, queries are limited throughout the application to the public graph and the private graph of the querying user, if authenticated.

The user can share a design by creating a share link, adding other SynBioHub users as owners of the design, and making the design public. When a share link is created, a one-way hash of the design URI and a secret value is used to ensure that share links can only be generated by a user who owns the design. If a share link is accessed, SynBioHub verifies the hash to make sure that the share link is valid. If it is, then the design is rendered for view-only. In order to allow another user access to edit the description, references, and other mutable fields, an owner of a design can add another owner to the design. Finally, once a design is complete and fully documented, the owner can make the design public, which moves the design into the public graph allowing it to be queried and viewed by anyone.

Designs can be shared not only with other users, but also with other applications. In particular, SynBioHub is able to communicate with Benchling and JBEI-ICE. SynBioHub can copy sequences to a repository within Benchling for sequence editing, as well as copy sequences back into SynBioHub collections. Similarly, designs can be transferred to/from JBEI-ICE. In both cases, these communications are enabled by translations to/from SBOL.

A design that is uploaded to a SynBioHub instance also becomes available to users from other instances due to

federation support in SynBioHub. Data federation is a technique that can be used to integrate data from remote sources using a single query.³³ For example, multiple RDF triplestores can be computationally queried using the SPARQL query language. Since SynBioHub is backed by an RDF triplestore, federation of queries to multiple SynBioHub instances is natively supported. SynBioHub uses SBOL as the common semantics to query and to create integrated views of designs for users.

Sharing designs between instances of SynBioHub is further facilitated via the Web of Registries service. Federated SynBioHub instances can reference parts in the public graph of other federated instances. In order to support this federation, the Web of Registries service maintains information about all SynBioHub registries, such as their name, description, administrator email, URI prefix, and uniform resource locator (URL) (Figure 3). It also keeps track of whether or not the

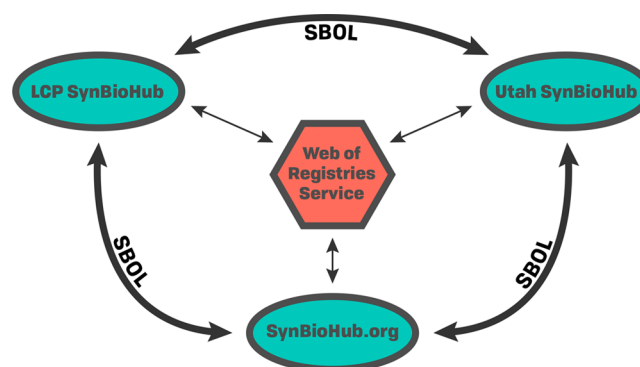


Figure 3. Web of Registries enables communication between SynBioHub instances. Any SynBioHub can access the Web of Registries to determine information about all registered SynBioHub instances. If a design references a part within another SynBioHub instance, the information about this part can be fetched in order to render this design information locally and provide links to the corresponding design information page for this part.

SynBioHub is being successfully updated. When a new SynBioHub instance wishes to join the Web of Registries, it sends its information to the Web of Registries service, and all the Web of Registries curators are alerted *via* email that there is a new repository pending approval. Once the repository has been approved, its information is broadcast to all other registries in the Web of Registries. Anybody can access the current list of registries through an HTTP GET request. Once registered, an instance of SynBioHub is able to locate designs within any other instance that is registered with the Web of Registries.

Computational API. In addition to providing a graphical user interface, SynBioHub also provides a computational API for programmatic access. The API is a RESTful Web service which can be used from any modern programming language, and has endpoints to search for, retrieve, and upload designs. Security is also a concern in the HTTP API. When a user is authenticated through the API, a token is returned. Any further queries that require authentication (submitting or searching nonpublic graphs) require that this token is passed in the X-*Authorization* header. On the backend, there is middleware which uses the authentication token to attach the user to the request in the same manner as requests from the SynBioHub user interface.

An interface to the API has also been implemented as a set of Java classes which have now been incorporated into libSBOLj³² (Figure 4). This integration of access to a SynBioHub

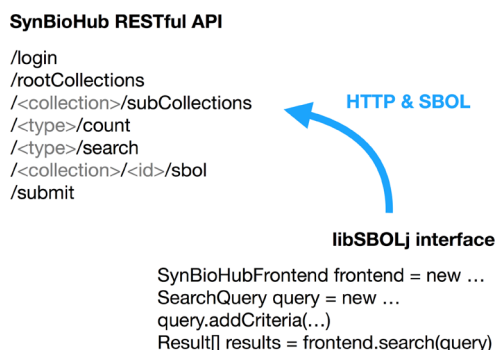


Figure 4. An example of some of the functionality that SynBioHub provides through its Application Programming Interface (API) and the relevant Java pseudocode. libSBOLj communicates with SynBioHub using the RESTful API over HTTP, and SBOL as the data exchange format.

repository has been tightly coupled with the SBOL library, and the coupling has been used to provide a novel system of completing partial designs upon load. If an SBOL file is loaded into libSBOLj and references URIs which are not present in the file but are present in a remote repository, libSBOLj can automatically retrieve the missing design information using the SynBioHub API to create a complete document. This automated resolution creates a system of “dependencies” as used in computer software libraries, but applied to biological designs.

The first software to utilize this libSBOLj interface to the API was SBOLDesigner, a sequence based computer-aided design tool.¹² This support allows users of SBOLDesigner to login to various instances of the repository and query, view, download, use, and upload designs. After logging in, designs from private collections can be seen and searched through. Any of these designs can be downloaded and inserted into the current design. From there, the design can be modified and used in various ways. After the sequence-level design is finished, the complete SBOL document can be uploaded back to SynBioHub into the user’s private collection.

Synthetic Biology Workflows. The architecture and features of SynBioHub enable several common workflows utilized by synthetic biologists to produce genetic design information. One such example workflow, depicted in Figure 5a, is the publication of sufficient information to enable reproducibility of genetic designs. In this workflow, a researcher begins by using their GDA tools of choice to create a genetic design. If this tool supports direct communication with the SynBioHub HTTP API, the parts created can be directly deposited in the SynBioHub repository. Otherwise, GenBank, FASTA, or SBOL files exported by the design software can be uploaded into SynBioHub. Once the design is uploaded, the author can obtain a sharable link that can be provided to the journal and used by reviewers to privately access the design information and evaluate its completeness as part of the review process. SynBioHub can also produce an SBOL Visual diagram that can be downloaded from the page and included in the publication, if desired. Once the paper has been accepted for publication, the design can then be moved into the public space on SynBioHub, and a link to the design can be included in the publication, granting readers direct access to all the information provided about the design.

Another example workflow is that of a team competing in the popular Internationally Genetically Engineered Machines (iGEM) competition (Figure 5b). One team member can create a design using iGEM parts and plasmids retrieved directly from SynBioHub using a sequence editing tool such as SBOLDesigner. The resulting composite design can then be uploaded directly to SynBioHub. The design information can be placed into a single collection for their project. Using SynBioHub’s ownership management functionality, they can add their teammates to this collection as owners, allowing them to modify this design. Another team member may produce an assembly plan using the Benchling tool by copying from SynBioHub the design information into the Benchling repository for the team. Another teammate may add additional information to the assembly plan using the free-text mutable part description fields on SynBioHub. Finally, in the future, SynBioHub could potentially communicate with the iGEM registry through its XML API, allowing push-button publication of the team’s data to the iGEM registry for the competition.

Discussion. The description and storage of information that allows for reproducible genetic designs and constructs is one of

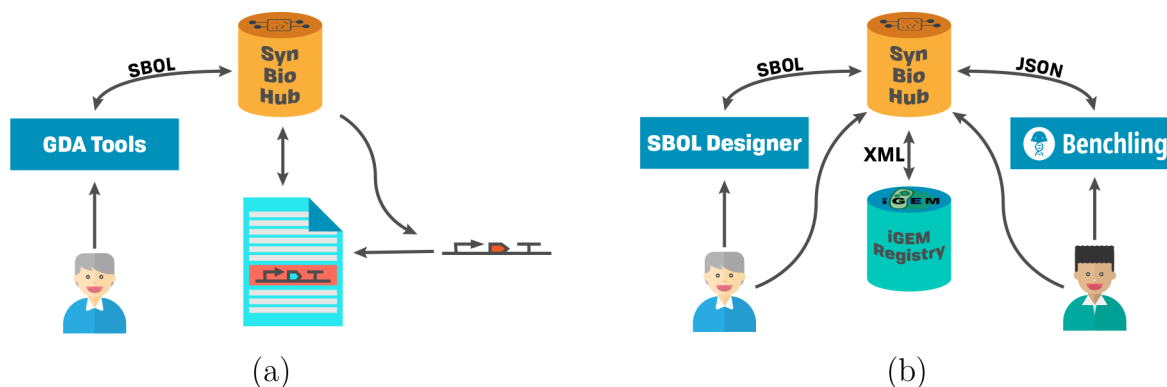


Figure 5. Synthetic biology workflows enabled by SynBioHub. (a) A research scientist prepares to publish information about genetic constructs. Constructs can be pushed to SynBioHub, where they can be referenced by a static URL in the published article. Additionally, SynBioHub can generate SBOL Visual figures (currently only in PNG format) for uploaded parts. (b) An iGEM team can utilize the sharing and ownership functionality of SynBioHub to enable multiple users to collaborate on the same design. API data transfers also support each step of the design process, allowing for data transfer without manual file download/upload.

the contemporary challenges of synthetic biology. The SynBioHub repository contributes to this endeavor by assisting in the sharing of such genetic designs and facilitating the integration of existing software packages and databases. The vision for SynBioHub is threefold. First, the system provides a user-facing repository for designs encoded in the emerging community standard SBOL. Second, the system supports the web of registries concept through data federation, while also acting as an information storage and retrieval system for supporting the operation of standardized workflows of synthetic biology tools. Third, the system can act as an integration hub for other types of biological information, including experimental data. The capabilities of SBOL to overcome the limited access to DNA sequence information displayed by other formats, such as GenBank, is key to the functioning of SynBioHub. SBOL provides detailed descriptions of not only sequences, but also other essential information such as interactions between components, hierarchies, and provenance of designs. Altogether, SBOL allows SynBioHub to structure the repository around an information-rich design format; a goal arguably unattained by any other synthetic biology design repository to date.

SBOL is being increasingly adopted by the synthetic biology community at a steady rate. However, SynBioHub provides several features that can potentially accelerate this process. First, it provides public access to SBOL encoded information, such as the iGEM dataset. Second, it provides an API for programmatic access by software tools to fetch and store genetic designs. Finally, it includes a means to communicate with software tools such as Benchling and JBEI ICE, allowing it to serve as a unifying hub for the development of synthetic biology workflows. Each of these features has been enabled by the ability of SBOL to store and represent genetic design information. SBOL 2, in particular, extends beyond the representation of just genetic features. The SBOL data model allows parts hierarchies, modules, and mathematical models to be stored, together with information about other molecules such as RNAs, proteins, metabolites, and the interactions between these molecules. This feature is essential to support a top-down approach to engineering biological systems where the design is carried out at the phenotype and pathway level and the genetics of the system is derived last of all.

In the future, we plan to extend the data available within SynBioHub instances *via* data integration.³⁴ Similar to the conversion that we performed of the iGEM data set, we would like to extract data from other repositories, such as UniProt, Reactome, and RegulonDB. Next, while experimental data can be provided using the attachment feature of SynBioHub, we plan to support the new experimental data extensions of SBOL to better link the designs to their implementations and the data generated through tests, enabling the documentation of the entire design-build-test process. In a similar fashion, the models and their simulation experimental data should also be better linked with their designs.

As the field of synthetic biology continues to develop, we are seeing an increasing number of different part collections with different functional characteristics. The idea of a single repository for all parts is no longer desirable or scalable. SynBioHub supports the federation of instances. This feature allows individual laboratories and/or research projects to install, and federate, their own SynBioHub repository, which will be associated with other instances in the Web of Registries. We advocate that the use of SynBioHub will improve the way

information on synthetic biology constructs is exchanged, and will thus help to overcome the reproducibility failures within a rapidly growing community.

Availability. The SynBioHub source code is available on GitHub under the BSD 2-clause license. SynBioHub is a freely available open source project; any institution or developer can host an instance of SynBioHub. An example instance of SynBioHub is provided at <http://synbiohub.org>, which includes the complete iGEM registry of standard biological parts (as of 2016) converted to SBOL format, and provides federated access to the public JBEI-ICE design repository. For programmatic access, communication with SynBioHub is provided as an integral part of the three major SBOL libraries: libSBOLj (Java), pysbol (Python), and libSBOL (C++). The use of the libSBOLj SynBioHub integration has been demonstrated in SBOLDesigner¹² and iBioSim¹³ tools to query to find existing parts in SynBioHub and upload new designs to SynBioHub.

METHODS

SynBioHub is a Node.js application backed by the Virtuoso RDF triplestore. SynBioHub takes advantage of the RDF semantics of SBOL. Although SBOL has typically been used as a file format, the SBOL data model is expressed in RDF and thus can be stored in a triplestore graph database. This approach was first used by the SBOL Stack,²¹ and has now been extended by SynBioHub to accommodate the entire SBOL data model. This provides a significant advantage in that SynBioHub does not need a custom data storage format; instead, SBOL can be used as-is and queried as a graph. Figure 6 shows an example of how SynBioHub queries the RDF triplestore to retrieve a list of SBOL collections in order to display the collections view.

```
PREFIX sbol2: <http://sbols.org/v2#>
PREFIX dcterms: <http://purl.org/dc/terms/>
PREFIX ncbi: <http://www.ncbi.nlm.nih.gov/#>

SELECT ?Collection ?name ?description ?displayId ?version WHERE {
  ?Collection a sbol2:Collection .
  FILTER NOT EXISTS { ?otherCollection sbol2:member ?Collection }
  OPTIONAL { ?Collection dcterms:title ?name . }
  OPTIONAL { ?Collection sbol2:displayId ?displayId . }
  OPTIONAL { ?Collection dcterms:description ?description . }
  OPTIONAL { ?Collection sbol2:version ?version }
}
```

Figure 6. An example of a SPARQL query used by SynBioHub to retrieve the list of root collections. The collections are represented in the triplestore using standard SBOL *Collection* entities. The query selects all collections, then filters for collections that are not contained by another collection. The name, displayId, description, and version attributes are retrieved if present.

AUTHOR INFORMATION

Corresponding Author

*E-mail: anil.wipat@ncl.ac.uk.

ORCID

Zach Zundel: 0000-0003-1355-6721

Michael Zhang: 0000-0002-1084-6734

Anil Wipat: 0000-0001-7310-4191

Author Contributions

J.A.M., G.M., I.D.O., and A.W. developed the initial version of the SynBioHub software previously described in the SBOL Stack paper. The aforementioned authors, alongside C.J.M., Z.Z., M.Z., and A.G.M., furthered development of SynBioHub as a stand-alone software project.

Notes

The authors declare no competing financial interest.

ACKNOWLEDGMENTS

The authors thank Dr. Nathan Hillson and Hector Plahar of JBEI for collaboration on the JBEI-ICE integration, and Randy Rettberg of iGEM Headquarters for providing computational access to the iGEM Parts Registry. The authors would also like to thank Professor Herbert Sauro, Dr. Bryan Bartley, and Kiri Choi for developing support within libSBOL and pySBOL to communicate with SynBioHub. The authors of this work are supported by The Engineering and Physical Sciences Research Council grants EP/J02175X/1 and EP/N031962/1 (J.A.M. and A.W.) and EP/R019002/1 (A.G.M.), and the National Science Foundation under Grant No. CCF-1218095 and DBI-1356041 (C.M., Z.Z., and M.Z.). G.M. is, in part, supported by the EPSRC grant EP/J02175X/1. J.A.M. is supported by FUJIFILM DioSynth Biotechnologies. Z.Z. has been supported by Google Summer of Code 2017.

REFERENCES

- Church, G. M., Elowitz, M. B., Smolke, C. D., Voigt, C. A., and Weiss, R. (2014) Realizing the potential of synthetic biology. *Nat. Rev. Mol. Cell Biol.* 15, 289.
- Myers, C. J., Beal, J., Gorochowski, T. E., Kuwahara, H., Madsen, C., McLaughlin, J. A., Misirlı, G., Nguyen, T., Oberortner, E., Samineni, M., Wipat, A., Zhang, M., and Zundel, Z. (2017) A standard-enabled workflow for synthetic biology. *Biochem. Soc. Trans.* 45, 793–803.
- Beal, J., Weiss, R., Densmore, D., Adler, A., Appleton, E., Babb, J., Bhatia, S., Davidsohn, N., Haddock, T., Loyall, J., Schantz, R., Vasilev, V., and Yaman, F. (2012) An end-to-end workflow for engineering of biological networks from high-level specifications. *ACS Synth. Biol.* 1, 317–331.
- Litcofsky, K. D., Afeyan, R. B., Krom, R. J., Khalil, A. S., and Collins, J. J. (2012) Iterative plug-and-play methodology for constructing and modifying synthetic gene networks. *Nat. Methods* 9, 1077–1080.
- Goni-Moreno, A., Carcajona, M., Kim, J., Martínez-García, E., Amos, M., and de Lorenzo, V. (2016) An implementation-focused bio/algorithmic workflow for synthetic biology. *ACS Synth. Biol.* 5, 1127–1135.
- Misirlı, G., Madsen, C., de Murieta, I. S., Bultelle, M., Flanagan, K., Pocock, M., Hallinan, J., McLaughlin, J. A., Clark-Casey, J., Lyne, M., Micklem, G., Stan, G.-B., Kitney, R., and Wipat, A. (2017) Constructing synthetic biology workflows in the cloud. *Engineering Biology* 1, 61–65.
- Andrianantoandro, E., Basu, S., Karig, D. K., and Weiss, R. (2006) Synthetic biology: new engineering rules for an emerging discipline. *Mol. Syst. Biol.*, DOI: 10.1038/msb4100073.
- MacDonald, J. T., Barnes, C., Kitney, R. I., Freemont, P. S., and Stan, G.-B. V. (2011) Computational design approaches and tools for synthetic biology. *Integrative Biology* 3, 97–108.
- Myers, C. J. (2013) *Microbial Synthetic Biology*, Vol. 40, pp 177–202, Academic.
- Otero-Muras, I., and Banga, J. R. (2017) Automated Design Framework for Synthetic Biology Exploiting Pareto Optimality. *ACS Synth. Biol.* 6, 1180.
- Chandran, D., Bergmann, F. T., and Sauro, H. M. (2009) TinkerCell: modular CAD tool for synthetic biology. *J. Biol. Eng.* 3, 19.
- Zhang, M., McLaughlin, J. A., Wipat, A., and Myers, C. J. (2017) SBOLDesigner 2: An Intuitive Tool for Structural Genetic Design. *ACS Synth. Biol.* 6, 1150.
- Madsen, C., Myers, C., Patterson, T., Roehner, N., Stevens, J., and Winstead, C. (2012) Design and Test of Genetic Circuits Using iBioSim. *IEEE Design & Test of Computers* 29, 32–39.
- Huynh, L., and Tagkopoulos, I. (2014) Optimal part and module selection for synthetic gene circuit design automation. *ACS Synth. Biol.* 3, 556–564.
- Nielsen, A. A., Der, B. S., Shin, J., Vaidyanathan, P., Paralanov, V., Strychalski, E. A., Ross, D., Densmore, D., and Voigt, C. A. (2016) Genetic circuit design automation. *Science* 352, aac7341.
- Bhatia, S., and Densmore, D. (2013) Pigeon: a design visualizer for synthetic biology. *ACS Synth. Biol.* 2, 348–350.
- Der, B. S., Glassey, E., Bartley, B. A., Enghuus, C., Goodman, D. B., Gordon, D. B., Voigt, C. A., and Gorochowski, T. E. (2017) DNAPlotlib: programmable visualization of genetic designs and associated data. *ACS Synth. Biol.* 6, 1115.
- McLaughlin, J. A., Pocock, M., Misirlı, G., Madsen, C., and Wipat, A. (2016) VisBOL: web-based tools for synthetic biology design visualization. *ACS Synth. Biol.* 5, 874–876.
- Peccoud, J., Anderson, J. C., Chandran, D., Densmore, D., Galdzicki, M., Lux, M. W., Rodriguez, C. A., Stan, G.-B., and Sauro, H. M. (2011) Essential information for synthetic DNA sequences. *Nat. Biotechnol.* 29, 22–22.
- Ham, T. S., Dmytriv, Z., Plahar, H., Chen, J., Hillson, N. J., and Keasling, J. D. (2012) Design, implementation and practice of JBEI-ICE: an open source biological part registry platform and tools. *Nucleic Acids Res.* 40, e141–e141.
- Madsen, C., McLaughlin, J. A., Misirlı, G., Pocock, M., Flanagan, K., Hallinan, J., and Wipat, A. (2016) The SBOL Stack: a platform for storing, publishing, and sharing synthetic biology designs. *ACS Synth. Biol.* 5, 487–497.
- Lu, G., and Moriyama, E. N. (2004) Vector NTI, a balanced all-in-one sequence analysis suite. *Briefings Bioinf.* 5, 378–388.
- Herscovitch, M., Perkins, E., Baltus, A., and Fan, M. (2012) Addgene provides an open forum for plasmid sharing. *Nat. Biotechnol.* 30, 316–317.
- Galdzicki, M., et al. (2014) The Synthetic Biology Open Language (SBOL) provides a community standard for communicating designs in synthetic biology. *Nat. Biotechnol.* 32, 545–550.
- Roehner, N., et al. (2016) Sharing structure and function in biological design with SBOL 2.0. *ACS Synth. Biol.* 5, 498–506.
- Beal, J., et al. (2016) Synthetic Biology Open Language (SBOL) Version 2.1.0. *J. Integr. Bioinf.* 13, 30–132.
- Lebo, T., Sahoo, S., McGuinness, D., Belhajjame, K., Cheney, J., Corsar, D., Garijo, D., Soiland-Reyes, S., Zednik, S., and Zhao, J. (2013) PROV-O: The PROV Ontology: W3C Recommendation 30, Lancaster University.
- Quinn, J. Y., et al. (2015) SBOL Visual: A Graphical Language for Genetic Designs. *PLoS Biol.* 13, 1–9.
- Wolstencroft, K., Krebs, O., Snoep, J. L., Stanford, N. J., Bacall, F., Golebiewski, M., Kuzyakiv, R., Nguyen, Q., Owen, S., Soiland-Reyes, S., Straszewski, J., van Niekerk, D. D., Williams, A. R., Malmström, L., Rinn, B., Müller, W., and Goble, C. A. (2017) FAIRDOMHub: a repository and collaboration environment for sharing systems biology research. *Nucleic Acids Res.* 45, D404–D407.
- Shadbolt, N., Berners-Lee, T., and Hall, W. (2006) The semantic web revisited. *IEEE intelligent systems* 21, 96–101.
- Eilbeck, K., Lewis, S. E., Mungall, C. J., Yandell, M., Stein, L., Durbin, R., and Ashburner, M. (2005) The Sequence Ontology: a tool for the unification of genome annotations. *Genome Biology* 6, R44.
- Zhang, Z., Nguyen, T., Roehner, N., Misirlı, G., Pocock, M., Oberortner, E., Samineni, M., Zundel, Z., Beal, J., Clancy, K., Wipat, A., and Myers, C. J. (2016) libSBOLj 2.0: A Java Library to Support SBOL 2.0. *IEEE Life Sci. Lett.* 1, 34–37.
- Belleau, F., Nolin, M.-A., Tourigny, N., Rigault, P., and Morissette, J. (2008) Bio2RDF: towards a mashup to build bioinformatics knowledge systems. *J. Biomed. Inf.* 41, 706–716.
- Misirlı, G., Hallinan, J., Pocock, M., Lord, P., McLaughlin, J. A., Sauro, H., and Wipat, A. (2016) Data integration and mining for synthetic biology design. *ACS Synth. Biol.* 5, 1086–1097.